

ShopFlow

Système de Gestion d'une Boutique en Ligne

Mini-Projet Backend Spring Boot 3 / Frontend Next.js

Enseignante : Dr. Ing. Ghada Feki
Année universitaire 2025 – 2026
Durée 3 semaines — Travail individuel

Étudiant	yosriii Travail individuel
Stack	Java 21 • Spring Boot 3.4.13 • PostgreSQL 18 / H2 • Spring Security 6 + JWT 0.12.6 Springdoc 2.8.13 • MapStruct 1.5.5 • Next.js 16.3.5 + TypeScript + Tailwind 4
Dépôt	ShopFlow/ (backend 67 fichiers, frontend 11 routes) Git: ShopFlow/.git — 3b7f7ba
Date	22 Septembre 2026 Soutenance 10 min (3 min démo + 3 min code + 4 min questions)

Rapport technique — 7 pages — Architecture, choix complexes, difficultés, répartition

Sommaire

1.	Schéma d'architecture technique (couches + flux)	p.2
2.	Choix d'implémentation les plus complexes	p.3-4
3.	Difficultés rencontrées et solutions	p.5
4.	Répartition des tâches	p.6
5.	Grille d'évaluation & mapping endpoints	p.6
6.	Installation & lancement	p.7
Annexe.	Postman + Swagger + Captures	p.7

1 Schéma d'architecture technique

Le projet respecte strictement l'architecture en couches imposée *Spec 3.2 p.4* : Controller → Service → Repository → Entity, avec DTO séparés et Mapper. Aucune entité n'est exposée directement.

Couche	Rôle	Technos	Exemples ShopFlow
Controller	Expose REST, @Valid, DTO + codes HTTP	@RestController, @RequestMapping	ProductController: GET /api/products?page&filtres;, POST @Valid
Service	Logique métier, @Transactional	Spring @Service	ProductService: stock, promo%, OrderService: PENDING→DELIVERED
Repository	Accès données, JPQL, Specifications, Pageable	JpaRepository, JpaRepositoryExecutor	ProductSpecifications: hasCategory, priceBetween, search(q)
Entity	JPA Jakarta 3, relations, lifecycle	Jakarta Persistence 3	12 entités: User, Product (M-M Category), Variant, Cart, Order...
DTO + Mapper	Request/Response séparés	MapStruct 1.5.5	ProductRequest → Product, Product → ProductResponse
Exception	Gestion globale erreurs	@ExceptionHandler	ResourceNotFoundException → 404, @Valid → 400
Security	JWT stateless, @PreAuthorize	Security 6 + jjwt 0.12.6	JwtService, JwtAuthenticationFilter, access 1h/refresh 7j

Flux de données — exemple création commande

Client (Next.js) → **POST /api/orders** avec *Bearer token* → **JwtAuthenticationFilter** vérifie signature + injecte SecurityContext → **OrderController @Valid** → **OrderService @Transactional** : vérifie stock final (Product + Variant), décrémente stock, calcule sousTotal/frais/totalTTC, applique Coupon, génère *ORD-2026-XXXXX* (@PrePersist), crée Order + OrderItems, vide Cart → **OrderRepository.save()** → Hibernate flush → **PostgreSQL (prod) / H2 (dev)** → retour **OrderResponse DTO 201** + Swagger.

Frontend consomme via **src/lib/api.ts** : *Axios* + *intercepteur* injecte Bearer sur chaque requête, intercepte 401 → POST /api/auth/refresh → rejoue requête (Spec 4.3). Guards Next.js protègent /cart, /orders, /seller.

```
■■■■■■■■■■ HTTP + Bearer ■■■■■■■■■■ @Transactional ■■■■■■■■■■
■ Next.js ■■■■■■■■■■ Spring Boot 3.4.13 ■■■■■■■■■■ PostgreSQL ■
■ (3000) ■■■■■■■■■■ (8080) ■■■■■■■■■■ / H2 (dev) ■
■ Axios ■ JSON DTO ■ Controller/Service ■ JPA/Hibernate ■
■ JWT Store ■ Pageable + @Valid ■ Repository + JWT ■ Specifications ■ 12 tables ■
```



Schéma simplifié — 3 tiers (Frontend → API → DB), stateless JWT, pagination obligatoire p.3.

2 Choix d'implémentation les plus complexes

2.1 Gestion du stock multi-niveaux (Product + Variant)

Problème : Spec 2.1 impose *variantes taille/couleur avec stock individuel + vérification temps réel panier/commande*. Solution : **Product.stock** (global) + **ProductVariant.stockSupplémentaire** + **prixDelta**. Dans *CartService.addItem()* et *OrderService.createOrder()*, stock disponible = product.stock + variant.stockSupplémentaire. Vérification à l'ajout panier ET à la création commande (dernière vérif p.3), décrémentation atomique en `@Transactional`. Si insuffisant → 400 *Stock insuffisant*. Choix Hibernate LAZY + orphanRemoval pour éviter N+1.

2.2 Filtrage & recherche avec Specifications + Pageable

Spec 2.1 exige *recherche plein texte nom/description/catégorie/prix min/max + tri + pagination obligatoire*. Au lieu de multiples méthodes JPQL, **ProductSpecifications.java** expose 6 Specifications combinables : `hasCategory`, `hasSeller`, `priceBetween`, `isPromo`, `isActive`, `search(q)`. Dans *ProductService.getProducts()* : `Specification.where(isActive(true)).and(hasCategory(...)).and(priceBetween(...))` + `productRepository.findAll(spec, pageable)`. Avantage : requête dynamique sans explosion de méthodes, compatible *JpaSpecificationExecutor*, testable, performant (LAZY). Fallback JPQL `search(q)` et `findTopSellingIds()` via *OrderItem GROUP BY*.

2.3 Numéro de commande unique & flux de statuts

Spec 2.4 : *ORD-2024-XXXXX + PENDING → PAID → PROCESSING → SHIPPED → DELIVERED + annulation seulement si PENDING/PAID + remboursement*. Implémentation : **Order.numeroCommande** généré via `@PrePersist generateNumero()` = `ORD-Year-5digits random + @Column(unique=true)`. Statut enum **OrderStatus** avec transition contrôlée dans *OrderService.updateStatus()* et *cancel()* (vérifie `if statut != PENDING && != PAID throw` + restock product.stock += quantité pour remboursement simulé). Chiffres logiques métier centralisés en Service, pas en Controller.

2.4 Sécurité JWT stateless access + refresh

Spec 3.1 : *Spring Security 6 + JWT jjwt access+refresh*. Choix : **jjwt 0.12.6** (patch CVE-2024-31033) + secret Base64 256 bits dans `application.yml`: `app.jwt.secret` (env `JWT_SECRET` en prod). **JwtService** génère access 1h + refresh 7j, signe HS384. **JwtAuthenticationFilter** (OncePerRequestFilter) extrait Bearer, valide signature + expiration, injecte *UsernamePasswordAuthenticationToken*. **SecurityConfig** stateless `SessionCreationPolicy.STATELESS`, permetAll sur `/api/products/**`, `/api/categories/**`, `/auth/**`, `/swagger-ui/**`, `/h2-console/**`, `authenticated()` ailleurs, `DaoAuthenticationProvider` + `BCryptPasswordEncoder`. Refresh : `POST /api/auth/refresh {refreshToken}` vérifie validité puis ré-émet access. Logout stateless → 200. Frontend **src/lib/api.ts** intercepte 401 → refresh → rejoue requête (Spec 4.3).

2.5 Frontend Next.js vs React pur & TypeScript

Spec 4.1 laisse choix *Next.js ou Angular*. Choix **Next.js 16.3.5** : framework React avec App Router (fichier → route), SSR/SSG pour SEO catalogue (*produits en vedette*), optimisation images, *server components*. React seul aurait nécessité *react-router* + config SSR manuelle. **TypeScript** obligatoire en pratique : types miroirs des DTO Spring Boot (Product, Cart, Order, Role) évitent erreurs *prix: number vs string*, autocomplétion, garde-fous pour *OrderStatusBadge*, *RatingStars* composants réutilisables (Spec 4.3).

2.6 DTO séparés + MapStruct (Spec 3.2)

Aucune entité exposée : **ProductRequest** (`@NotBlank nom`, `@NotNull prix/stock`) vs **ProductResponse** (+ `sellerNom`, `categories Set`, `noteMoyenne`, `pourcentageRemise calculé`). **ProductMapper**

@Mapper(componentModel="spring") gère toEntity / toResponse + toCategoryDtoSet. Lombok @Data/@Builder évite boilerplate. GlobalExceptionHandler centralise 400 (@Valid) + 404.

2.7 Base double PostgreSQL/H2 + pg_hba

Spec 3.1 : PostgreSQL prod / H2 dev. application.yml avec 3 profils : active: dev par défaut → jdbc:h2:mem:shopflow + ddl-auto: create-drop + H2 console enable. Profil prod → jdbc:postgresql://localhost:5432/shopflow + HikariCP + dialect PostgreSQLDialect + env DB_USERNAME/PASSWORD. Passage en prod : DB_USERNAME=shopflow DB_PASSWORD=shopflow mvn spring-boot:run -Dspring-boot.run.profiles=prod. Problème Fedora peer vs md5 résolu (voir §3).

3 Difficultés rencontrées et solutions

CVE Mend.io (logback 1.5.11, tomcat 10.1.31, js 2.17... 9.8)

Problème: Scanner Mend flaggeait 9.8 CVEs sur Spring Boot 3.3.5 (tomcat 10.1.31, js 2.17, logback 1.5.11...).

Solution: Upgrade 3.3.5 → 3.4.13 + overrides postgresql 42.7.7, jackson 2.20.2, logback 1.5.18, commons-lang3 3.20.0 → BUILD SUCCESS, restant 2026-* = false positives *Insufficient Information* documentés.

pg_hba.conf Ident auth (Fedora 44)

Problème: FATAL: Ident authentication failed pour shopflow/localhost après dnf install postgresql-server 18.

Solution: Passage host all all 127.0.0.1/32 md5 + local peer gardé, systemctl restart postgresql, puis CREATE USER shopflow → PGPASSWORD=shopflow psql -h localhost → 1. pgAdmin 9.17 → Host **localhost** (pas shopflow@localhost:5432).

Java 25 par défaut vs 21 LTS

Problème: Fedora 44 installait OpenJDK 25 (too new pour Boot 3.4), mvn -version not found.

Solution: Utilisation du JDK IntelliJ ms-21.0.12.1 (/home/yosriii/.jdk/ms-21.0.12.1) + JAVA_HOME + Maven embarqué Toolbox/.../maven3/bin/mvn. pom java.version 21.

printf() redondant + typos

Problème: IntelliJ flaggeait printf("Hello") sans args + 10 typos français dans pom.

Solution: Remplacement printf → println + traduction commentaires pom en anglais, BUILD SUCCESS 67 files.

Security 401 sur /api/products

Problème: Après ajout Spring Security, GET /api/products → 401 malgré permitAll initial.

Solution: Création **SecurityConfig** avec SecurityFilterChain stateless, permitAll explicite sur products/categories/auth/swagger/h2, ajout JwtAuthenticationFilter avant UsernamePasswordAuthenticationFilter → 200.

H2 create-drop efface données

Problème: Chaque mvn spring-boot:run drop/recreate → review test échouait (product not found).

Solution: Compris normal en dev (ddl-auto: create-drop); en soutenance utiliser prod ou update pour persistance. Data seeded via POST /api/categories + /products dans Postman.

4 Répartition des tâches

Travail **individuel** (yosriii) — 3 semaines découpées selon ordre méthodologique Spec p.7 :

Semaine 1	Modélisation + Config	pom.xml 3.4.13, application.yml (dev H2/prod PG), ShopFlowApplication, 12 Entity + 3 Enum, 12 Repository + Specifications
Semaine 2	Métier + API	DTO/MapStruct, Product/Category Services + Controllers, Cart (stock temps réel, coupon), Order (ORD-*, flux statuts, stock final), Auth JWT
Semaine 3	Finition + Frontend	Review/Coupon/Dashboard, SecurityConfig final, Swagger, Postman collection (20 req), Next.js 11 routes + intercepteur JWT, tests, rapport

Si binôme : *Backend (15 pts) : Spring Boot + JPA + Security (Dev A) ; Frontend (5 pts) : Next.js + Postman + Rapport (Dev B)* — ici tout réalisé solo, choix justifié par stack homogène Java 21.

5 Grille d'évaluation & mapping endpoints

Mapping Spec p.5 → implémentation et points p.6 :

Critère	Ce que nous évaluons	Réalisé
Modèle données JPA 3 pts	12 entités + relations M-M, 1-M, 1-1, LAZY, cascade, @CreationTimestamp	Fait (67 fichiers)
Repository 2 pts	JpaRepository + JPQL + Specifications + Pageable	ProductSpecifications 6 filtres + Pageable
Service 3 pts	Stock, coupon, statut, @Transactional	CartService, OrderService ORD-* + restock
Controller 2 pts	Codes HTTP 201/400/404, @Valid	ProductController, CartController... + GlobalExceptionHandler
Sécurité 3 pts	JWT access+refresh, @PreAuthorize	JwtService 0.12.6, SecurityConfig stateless
Qualité 2 pts	Lombok, Swagger, README	Swagger /swagger-ui.html, README + Postman
Bonus +2	JUnit >=70%	À compléter (starter-test prêt)
Frontend 5 pts	Next.js, JWT intercepteur, catalogue/panier/commandes	11 routes, ProductCard, api.ts refresh auto

Endpoints couverts p.5: Auth (4), Products (6), Categories (4), Cart (6), Orders (6), Reviews (3), Coupons (4), Dashboard (2) = 35 endpoints, 20 dans Postman.

6 Installation & lancement

Pré-requis : JDK 21 (ms-21.0.12.1), Maven 3.9.16 (Toolbox), Node 22, PostgreSQL 18.

Backend

```
1. git clone <repo> && cd ShopFlow
2. JAVA_HOME=/home/yosriii/.jdk/ms-21.0.12.1 mvn compile → BUILD SUCCESS
3. Dev : mvn spring-boot:run → http://localhost:8080/swagger-ui.html + /h2-console (jdbc:h2:mem:shopflow sa)
4. Prod : sudo systemctl start postgresql + DB_USERNAME=shopflow DB_PASSWORD=shopflow mvn spring-boot:run -Dspring-boot.run.profiles=prod
5. Tests rapides : curl -X POST /api/auth/register -d '{...}' → accessToken
```

Frontend

```
1. cd frontend && echo "NEXT_PUBLIC_API_URL=http://localhost:8080" > .env.local
2. npm install && npm run dev → http://localhost:3000
3. npm run build → 11 routes static
```

Démo soutenance 3 min : 1. Register SELLER → 2. POST /categories + POST /products (Seller) → 3. Register CUSTOMER → 4. Add to cart → 5. POST /orders → 6. GET /orders/my → 7. PUT /cancel (si PENDING) → 8. Dashboard admin.

Annexe — Swagger + Postman + Captures

Swagger UI accessible sur **http://localhost:8080/swagger-ui.html** via *springdoc-openapi-starter-webmvc-ui* 2.8.13 (Spec 3.1). Collection Postman : **shopflow.postman_collection.json** (20 requêtes, variables `{{base}}` + `{{accessToken}}`). Livrable ZIP : backend/ (sans target/), frontend/ (sans node_modules/.next), shopflow.postman_collection.json, README.md, rapport.pdf.

Ressources : Spring Boot docs, Spring Security 6, Lombok, MapStruct, Next.js docs (Spec p.8).

ShopFlow — yosrii — 22 Septembre 2026 — Git 3b7f7ba — 67 fichiers backend + 11 routes frontend — BUILD SUCCESS